

1. DPLYR — Cruce de bases y KPIs por segmento

Tienes dos data frames en R: `polizas` (pólizas emitidas) y `siniestros` (siniestros registrados). La relación es 1-a-N: una póliza puede tener cero o varios siniestros.

Estructura de los datos

`polizas` ($n = 8$ filas):

id_poliza	segmento	año	exposicion
101	PyME	2023	1.0
102	Corp	2023	2.0
103	PyME	2023	0.5
104	Corp	2024	1.5
105	PyME	2024	1.0
106	Corp	2024	2.0
107	PyME	2024	1.0
108	Corp	2024	1.5

`siniestros` ($n = 11$ filas):

id_poliza	monto_siniestro
101	1.200.000
101	850.000
103	600.000
105	2.100.000
106	430.000
107	980.000
107	1.350.000
108	750.000
108	1.600.000
108	920.000
108	410.000

Escribe el código R (usando `dplyr`) que produzca una tabla resumen **por segmento** con las siguientes métricas:

- Número de pólizas y exposición total
- Número de siniestros y monto total siniestrado
- Frecuencia de siniestros (siniestros / pólizas)
- Severidad media por siniestro
- Prima pura de riesgo estimada (frecuencia $\times$ severidad)
- Loss Ratio si la prima media por póliza es \$800.000

Solución

Primero se resumen los siniestros por póliza. Luego ese resumen se cruza con la base de pólizas mediante un `left_join`; así, las pólizas sin siniestros permanecen en el cálculo y no se duplican sus exposiciones.

```
library(dplyr)
```

```

polizas <- read.csv("C:/Users/User/Desktop/Archivos/polizas.txt", sep="|")
siniestros <- read.csv("C:/Users/User/Desktop/Archivos/siniestros.txt", sep="|")
siniestros_por_poliza <- siniestros %>%
  group_by(id_poliza) %>%
  summarise(
    n_siniestros = n(),
    monto_siniestros = sum(monto_siniestro),
    .groups = "drop"
  )

resumen_segmento <- polizas %>%
  left_join(siniestros_por_poliza, by = "id_poliza") %>%
  mutate(
    n_siniestros = coalesce(n_siniestros, 0L),
    monto_siniestros = coalesce(monto_siniestros, 0)
  ) %>%
  group_by(segmento) %>%
  summarise(
    n_polizas = n_distinct(id_poliza),
    exposicion_total = sum(exposicion),
    n_siniestros = sum(n_siniestros),
    monto_total = sum(monto_siniestros),
    frecuencia = n_siniestros / n_polizas,
    severidad_media = monto_total / n_siniestros,
    prima_pura = frecuencia * severidad_media,
    prima_total = n_polizas * 800000,
    loss_ratio = monto_total / prima_total,
    .groups = "drop"
  )

resumen_segmento

```

El resultado esperado es:

Segmento	Pólizas	Exposición	Siniestros	Monto total	Frecuencia	Severidad	Loss Ratio
Corp	4	7.0	5	\$4.110.000	1.25	\$822.000	128.44%
PyME	4	3.5	6	\$7.080.000	1.50	\$1.180.000	221.25%

La prima pura estimada es \$1.027.500 para Corp y \$1.770.000 para PyME.

2. DPLYR + TIDYR — Construcción de triángulo de desarrollo

En la base de datos de siniestros de la entidad se encuentran registros individuales de pago. Cada fila es un pago parcial de un siniestro. Se debe construir un **triángulo de desarrollo acumulado** a partir de estos datos crudos, listo para pasarlo a ChainLadder.

Datos individuales: pagos_siniestros

id_siniestro	año_ocurrencia	año_pago	monto_pago
S001	2021	2021	400.000
S001	2021	2022	280.000
S002	2021	2021	950.000
S003	2022	2022	620.000
S003	2022	2023	310.000
S004	2022	2022	480.000
S005	2023	2023	730.000
S006	2023	2023	390.000
S006	2023	2024	195.000
S007	2024	2024	850.000

La columna `año_desarrollo` se define como

$$\text{año_pago} - \text{año_ocurrencia} + 1.$$

Se requiere calcular el año de desarrollo, agregar los pagos por año de ocurrencia y desarrollo, acumularlos, pivotar el resultado al formato de triángulo y convertirlo en un objeto `triangle` mediante `as.triangle()`.

Solución

```
library(dplyr)
library(tidyr)
library(ChainLadder)

triangulo_df <- pagos_siniestros %>%
  mutate(
    año_desarrollo = año_pago - año_ocurrencia + 1
  ) %>%
  group_by(año_ocurrencia, año_desarrollo) %>%
  summarise(
    pago_incremental = sum(monto_pago),
    .groups = "drop"
  ) %>%
  group_by(año_ocurrencia) %>%
  arrange(año_desarrollo, .by_group = TRUE) %>%
```

```

mutate(
  pago_acumulado = cumsum(pago_incremental)
) %>%
ungroup() %>%
select(año_ocurrencia, año_desarrollo, pago_acumulado) %>%
pivot_wider(
  names_from = año_desarrollo,
  values_from = pago_acumulado
) %>%
arrange(año_ocurrencia)

matriz_triangulo <- as.matrix(triangulo_df[, -1])
rownames(matriz_triangulo) <- triangulo_df$año_ocurrencia

triangulo_chainladder <- as.triangle(matriz_triangulo)
triangulo_chainladder

```

El triángulo acumulado esperado, expresado en miles, es:

Año de ocurrencia	Desarrollo 1	Desarrollo 2
2021	1.350	1.630
2022	1.100	1.410
2023	1.120	1.315
2024	850	NA

3. ChainLadder — Método Mack y estimación de IBNR

Con el triángulo acumulado ya construido, se aplica el método de **Mack Chain Ladder** para estimar las reservas IBNR de la cartera de accidentes personales. El triángulo tiene cinco años de origen (2020–2024) y cinco períodos de desarrollo.

Triángulo de pagos acumulados (COP millones)

Año	Dev 1	Dev 2	Dev 3	Dev 4	Dev 5
2020	1.200	1.560	1.680	1.740	1.760
2021	1.350	1.755	1.890	1.958	NA
2022	1.500	1.950	2.100	NA	NA
2023	1.650	2.145	NA	NA	NA
2024	1.800	NA	NA	NA	NA

Se requiere construir el triángulo como objeto R, aplicar `MackChainLadder()` con estimación de sigma, extraer los factores de desarrollo y el IBNR con su intervalo de confianza, e identificar el año de origen con mayor incertidumbre relativa.

Solución

```
library(ChainLadder)
library(dplyr)

# as.triangle -----
matriz <- matrix(
  c(
    1200, 1560, 1680, 1740, 1760,
    1350, 1755, 1890, 1958, NA,
    1500, 1950, 2100, NA, NA,
    1650, 2145, NA, NA, NA,
    1800, NA, NA, NA, NA
  ),
  nrow = 5,
  byrow = TRUE,
  dimnames = list(
    origen = 2020:2024,
    desarrollo = paste0("Dev_", 1:5)
  )
)
triangulo <- as.triangle(matriz)

# Modelo mack -----
```

```

modelo_mack <- MackChainLadder(
  triangulo,
  est.sigma = "Mack"
)

# Factores de desarrollo ata -----
ldf <- modelo_mack$f
ldf
# IBNR y error estándar por año de origen ####
resumen_mack <- summary(modelo_mack)$ByOrigin
resultado_origen <- data.frame(
  año_origen = as.integer(rownames(resumen_mack)),
  IBNR       = as.numeric(resumen_mack$IBNR),
  Mack_SE    = as.numeric(resumen_mack$Mack.S.E)
) %>%
  mutate(
    limite_inferior_95 = pmax(0, IBNR - 1.96 * Mack_SE),
    limite_superior_95 = IBNR + 1.96 * Mack_SE,
    CV = if_else(IBNR > 0, Mack_SE / IBNR, NA_real_)
  )
resultado_origen
# IBNR total e intervalo de confianza aproximado al 95 % -----
ibnr_total <- sum(summary(modelo_mack)$ByOrigin$IBNR)
se_total <- modelo_mack$Total.Mack.S.E
intervalo_total <- c(
  inferior = max(0, ibnr_total - 1.96 * se_total),
  estimacion = ibnr_total,
  superior = ibnr_total + 1.96 * se_total
)

# Año con mayor incertidumbre relativa -----
mayor_incertidumbre <- resultado_origen %>%
  filter(!is.na(CV)) %>%
  slice_max(CV, n = 1, with_ties = FALSE)
resultado_origen
intervalo_total
mayor_incertidumbre

```

Los factores de desarrollo calculados son aproximadamente:

Factor	$f_{1 \rightarrow 2}$	$f_{2 \rightarrow 3}$	$f_{3 \rightarrow 4}$	$f_{4 \rightarrow 5}$
LDF	1.3000	1.0769	1.0359	1.0115

Las estimaciones puntuales del IBNR, en COP millones, son:

Año de origen	Último acumulado	IBNR estimado
2020	1.760	0.00
2021	1.958	22.51
2022	2.100	100.30
2023	2.145	275.33
2024	1.800	840.36
Total		1.238,49

Resultados obtenidos

La tabla `resultado_origen` obtenida en R es:

año_origen	IBNR	Mack_SE	limite_inferior_95	limite_superior_95	CV
2020	0.00000	0.000000e+00	0.00000	0.00000	NA
2021	22.50575	4.583586e-13	22.50575	22.50575	2.036629e-14
2022	100.29750	4.608824e-01	99.39417	101.20083	4.595153e-03
2023	275.32725	4.922476e-01	274.36244	276.29205	1.787864e-03
2024	840.35700	5.232363e-01	839.33145	841.38254	6.226357e-04

El intervalo aproximado al 95% para el IBNR total es:

Inferior	Estimación	Superior
1.236,254	1.238,487	1.240,721

El año de origen con mayor incertidumbre relativa es 2022:

Año	IBNR	Mack SE	Límite inferior	Límite superior	CV
2022	100.2975	0.4608824	99.39417	101.2008	0.004595153

Aunque el año 2024 tiene un error estándar absoluto ligeramente mayor (0.5232 vs 0.4608 de 2022), la reserva IBNR de 2024 es gigantesca (840.35). En cambio, la reserva de 2022 es mucho más pequeña (100.29). Por lo tanto, con respecto al tamaño de su propia reserva, el año 2022 está expuesto a una mayor volatilidad proporcional que los años más recientes.

4. Ajuste de distribución de severidad

Se recibe un vector de montos de siniestros individuales, expresados en COP miles, de una cartera de accidentes personales. Se debe determinar qué distribución modela mejor la severidad para utilizarla en el pricing GLM y en el cálculo de una capa de reaseguro XL.

Siniestros individuales

```
siniestros <- c(
  120, 450, 820, 230, 1200, 340, 95, 2100, 510, 670,
  180, 340, 890, 1500, 290, 440, 760, 1100, 380, 620
)
```

Se requiere ajustar las distribuciones LogNormal, Pareto y Gamma; comparar los ajustes mediante AIC, BIC y el estadístico de Kolmogorov–Smirnov; y, con el mejor modelo, estimar la pérdida esperada de la capa XL \$500K *vs* \$500K.

Solución

```
library(fitdistrplus)
library(actuar)
library(dplyr)
siniestros <- c(
  120, 450, 820, 230, 1200, 340, 95, 2100, 510, 670,
  180, 340, 890, 1500, 290, 440, 760, 1100, 380, 620
)

# Ajuste por máxima verosimilitud -----

ajuste_lognormal <- fitdist(siniestros,"lnorm")

ajuste_pareto <- fitdist(
  siniestros,
  "pareto",
  start = list(shape=1, scale=min(siniestros)*0.95),
  lower = c(1e-8,1e-8),
  upper = c(Inf,min(siniestros))
)

ajuste_gamma <- fitdist(siniestros,"gamma")

# Estadístico KS para comparar los ajustes -----
```

```

ks_stat <- function(x,p_acumulada){
  x <- sort(x)
  n <- length(x)
  Fx <- p_acumulada(x)
  max(c(seq_len(n) / n - Fx, Fx - (seq_len(n) - 1) / n))
}
# Que tan bien se ajusta a una lognormal
ks_lognormal <- ks_stat(
  siniestros,
  function(x) plnorm(
    x,
    meanlog = ajuste_lognormal$estimate["meanlog"],
    sdlog = ajuste_lognormal$estimate["sdlog"]
  )
)

ks_pareto <- ks_stat(
  siniestros,
  function(x) ppareto(
    x,
    shape = ajuste_pareto$estimate["shape"],
    scale = ajuste_pareto$estimate["scale"]
  )
)

ks_gamma <- ks_stat(
  siniestros,
  function(x) pgamma(
    x,
    shape = ajuste_gamma$estimate["shape"],
    rate = ajuste_gamma$estimate["rate"]
  )
)

n <- length(siniestros)

tabla_ajustes <- data.frame(
  model = c("Lognormal","Pareto","Gamma"),
  AIC = c(
    ajuste_lognormal$aic,
    ajuste_pareto$aic,
    ajuste_gamma$aic
  ),
  BIC = c(
    -2*ajuste_lognormal$loglik + length(ajuste_lognormal$estimate)*log(n),
    -2*ajuste_pareto$loglik + length(ajuste_pareto$estimate)*log(n),

```

```

      -2*ajuste_gamma$loglik + length(ajuste_pareto$estimate)*log(n)
    ),
    KS = c(ks_lognormal,ks_pareto,ks_gamma)

) %>%
  arrange(AIC)
tabla_ajustes

# Selección automática del modelo con menor AIC -----
mejor_modelo <- tabla_ajustes[1,1]
cdf_mejor <- switch(
  mejor_modelo,
  Lognormal = function(x) plnorm(
    x,
    meanlog = ajuste_lognormal$estimate["meanlog"],
    sdlog = ajuste_lognormal$estimate["sdlog"]
  ),
  Pareto = function(x) ppareto(
    x,
    shape = ajuste_pareto$estimate["shape"],
    scale = ajuste_pareto$estimate["scale"]
  ),
  Gamma = function(x) pgamma(
    x,
    shape = ajuste_gamma$estimate["shape"],
    rate = ajuste_gamma$estimate["rate"]
  )
)

# E[min((X - 500)+, 500)] = integral entre 500 y 1000 de S(x) dx -----

perdida_esperada_capa <- integrate(
  function(x) 1 - cdf_mejor(x),
  lower = 500,
  upper = 1000
)$value
mejor_modelo
perdida_esperada_capa

```

Los resultados aproximados de la comparación son:

Modelo	AIC	BIC	KS
Lognormal	299.2160	301.2074	0.075072411
Gamma	299.2761	301.2676	0.08931328
Pareto	325.3536	327.3451	0.33515175

La Lognormal presenta el menor AIC, el menor BIC y el menor estadístico KS, por lo que es el mejor ajuste entre las tres alternativas. Con este modelo, la pérdida esperada de la capa \$500K *vs* \$500K es aproximadamente 154.0123 COP miles por siniestro.

5. Modelo de tarificación Poisson con offset

Se construye un GLM Poisson para modelar la frecuencia de siniestros de una cartera de accidentes personales. El conjunto de datos contiene 12 grupos tarifarios definidos por edad y actividad, junto con su exposición y el número de siniestros observados.

Dataset: `datos_ap`

Edad	Actividad	Exposición	Siniestros
18-30	alto	1.00	3
18-30	bajo	0.50	0
31-45	bajo	1.00	1
31-45	alto	2.00	5
46-60	bajo	1.00	2
18-30	alto	0.25	0
46-60	alto	1.50	4
31-45	bajo	1.00	1
60+	alto	3.00	7
60+	bajo	0.75	2
46-60	alto	2.00	6
60+	alto	1.00	3

Se requiere ajustar el GLM Poisson con offset de la exposición, interpretar los coeficientes como relatividades tarifarias, construir tablas por nivel, predecir la frecuencia de un asegurado de 55 años con actividad alta y un año de exposición, y evaluar la sobredispersión.

Solución

```
library(dplyr)

datos_ap <- data.frame(
  edad = factor(
    c(
      "18-30", "18-30", "31-45", "31-45",
      "46-60", "18-30", "46-60", "31-45",
      "60+", "60+", "46-60", "60+"
    )
  ),
```

```

    levels = c("18-30", "31-45", "46-60", "60+")
  ),
  actividad = factor(
    c(
      "alto", "bajo", "bajo", "alto", "bajo", "alto",
      "alto", "bajo", "alto", "bajo", "alto", "alto"
    ),
    levels = c("alto", "bajo")
  ),
  exposicion = c(1, 0.5, 1, 2, 1, 0.25, 1.5, 1, 3, 0.75, 2, 1),
  n_siniestros = c(3, 0, 1, 5, 2, 0, 4, 1, 7, 2, 6, 3)
)

# GLM Poisson con offset -----
modelo_poisson <- glm(
  n_siniestros ~ edad + actividad + offset(log(exposicion)),
  family = poisson(link = "log"),
  data = datos_ap
)

summary(modelo_poisson)

# Coeficientes y relatividades tarifarias -----
coeficientes <- data.frame(
  termino = names(coef(modelo_poisson)),
  beta = unname(coef(modelo_poisson)),
  relatividad = unname(exp(coef(modelo_poisson)))
)

tasa_base <- exp(coef(modelo_poisson)["(Intercept)"])

tabla_edad <- data.frame(
  variable = "edad",
  nivel = c("18-30", "31-45", "46-60", "60+"),
  relatividad = c(
    1,
    exp(coef(modelo_poisson)["edad31-45"]),
    exp(coef(modelo_poisson)["edad46-60"]),
    exp(coef(modelo_poisson)["edad60+"])
  )
) %>%
  mutate(tasa = tasa_base * relatividad)

tabla_actividad <- data.frame(
  variable = "actividad",
  nivel = c("alto", "bajo"),
  relatividad = c(
    1,

```

```

    exp(coef(modelo_poisson)["actividadbajo"])
  )
) %>%
  mutate(tasa = tasa_base * relatividad)

tabla_tarifaria <- bind_rows(tabla_edad, tabla_actividad)
rownames(tabla_tarifaria) <- NULL
coeficientes
tabla_tarifaria

# Predicción: 55 años, actividad alta y exposición de un año -----
nuevo_asegurado <- data.frame(
  edad = factor("46-60", levels = levels(datos_ap$edad)),
  actividad = factor("alto", levels = levels(datos_ap$actividad)),
  exposicion = 1
)

frecuencia_esperada <- predict(
  modelo_poisson,
  newdata = nuevo_asegurado,
  type = "response"
)

# Bondad de ajuste y prueba de sobredispersión -----
deviance_residual <- deviance(modelo_poisson)
grados_libertad <- df.residual(modelo_poisson)
p_deviance <- pchisq(
  deviance_residual,
  df = grados_libertad,
  lower.tail = FALSE
)

chi_pearson <- sum(residuals(modelo_poisson, type = "pearson")^2)
dispersion <- chi_pearson / grados_libertad
p_sobredispersión <- pchisq(
  chi_pearson,
  df = grados_libertad,
  lower.tail = FALSE
)

frecuencia_esperada
c(
  deviance = deviance_residual,
  gl = grados_libertad,
  p_deviance = p_deviance,
  dispersion = dispersion,
  p_sobredispersión = p_sobredispersión
)

```

Los coeficientes y relatividades aproximadas son:

Término	Coeficiente	tasa base
Intercepto	0.6675	1.9494
Edad 31–45	0.1292	1.1379
Edad 46–60	0.4118	1.5095
Edad 60+	0.3282	1.3885
Actividad baja	−0.5485	0.5778

La categoría base es una persona de 18–30 años con actividad alta, cuya tasa estimada es 1.9494 siniestros por año de exposición. Para una persona de 55 años, actividad alta y exposición de un año, la frecuencia esperada es aproximadamente 2.9428 siniestros.

La deviance residual es 3.5547 con 7 grados de libertad ($p \approx 0.8294$). El estadístico de dispersión de Pearson es 0.3822 y el valor p de la prueba es aproximadamente 0.9133; por tanto, no hay evidencia de sobredispersión.

6. LIFECONTINGENCIES — Prima neta de seguro temporal de vida

Se desea lanzar un seguro de vida temporal a 20 años. Utilizando el paquete `lifecontingencies` y una tabla de mortalidad incorporada, se calcula la prima neta anual y se analiza su sensibilidad frente a la tasa de interés técnico.

Parámetros del producto

Parámetro	Valor
Edad del asegurado	$x = 40$ años
Plazo	$n = 20$ años
Suma asegurada	COP \$100.000.000
Tasas evaluadas	3%, 4%, 5% y 6%
Pago del siniestro	Fin del año de fallecimiento
Pago de primas	Anual anticipado mientras sobreviva

Sea $v = (1 + i)^{-1}$ el factor anual de descuento. Para una persona de edad x , el valor actuarial de un seguro temporal de n años, con beneficio unitario pagadero al final del año de fallecimiento, se expresa como

$$A_{x:\overline{n}|}^1(i) = \sum_{k=0}^{n-1} v^{k+1} {}_k p_x q_{x+k}.$$

El valor actuarial de una anualidad temporal anticipada de cuantía unitaria, pagadera mientras el asegurado permanezca con vida, es

$$\ddot{a}_{x:\overline{n}|}(i) = \sum_{k=0}^{n-1} v^k {}_k p_x.$$

Por el principio de equivalencia, el valor presente esperado de las primas debe ser igual al valor presente esperado del beneficio:

$$P(i) \ddot{a}_{40:\overline{20}|}(i) = SA A_{40:\overline{20}|}^1(i).$$

Por tanto, la prima neta anual nivelada es

$$P(i) = SA \frac{A_{40:\overline{20}|}^1(i)}{\ddot{a}_{40:\overline{20}|}(i)}.$$

En esta notación, $A_{40:\overline{20}|}^1(i)$ representa el costo actuarial por unidad de suma asegurada y $\ddot{a}_{40:\overline{20}|}(i)$ representa el valor actuarial de la corriente de primas anuales anticipadas.

Solución

```
library(lifecontingencies)

# Tabla incorporada basada en la ley de Makeham -----
data("soa08Act", package = "lifecontingencies")
tabla_mortalidad <- soa08Act

# Parámetros del producto -----
edad <- 40
plazo <- 20
suma_asegurada <- 100000000
tasas <- c(0.03, 0.04, 0.05, 0.06)

# Seguro temporal, anualidad anticipada y prima neta -----
# m = 0 el periodo empieza a cubrir de inmediato
# k = 1 el beneficio se paga al final del año del fallecimiento (frecuencia anual)
resultados_prima <- do.call(
  rbind,
  lapply(tasas, function(tasa) {
    valor_seguro <- Axn(
      actuarialtable = tabla_mortalidad,
      x = edad,
      n = plazo,
      i = tasa,
      m = 0,
      k = 1
    )

    valor_anualidad <- axn(
      actuarialtable = tabla_mortalidad,
      x = edad,
      n = plazo,
      i = tasa,
      m = 0,
      k = 1,
      payment = "advance"
    )

    prima_neta <- suma_asegurada * valor_seguro / valor_anualidad

    data.frame(
      tasa = tasa,
      Axn = valor_seguro,
      axn = valor_anualidad,
      prima_neta_anual = prima_neta
    )
  })
)
```

```

    })
  )

resultados_prima

# Sensibilidad de la prima frente a la tasa de interés -----
plot(
  resultados_prima$tasa * 100,
  resultados_prima$prima_neta_anual,
  type = "b",
  pch = 19,
  col = "steelblue",
  xlab = "Tasa de interés técnico (%)",
  ylab = "Prima neta anual (COP)",
  main = "Sensibilidad de la prima neta anual"
)
grid()

```

Los resultados aproximados son:

Tasa	$A_{40:\overline{20} }^1$	$\ddot{a}_{40:\overline{20} }$	Prima neta anual
3%	0.08365	14.74815	COP \$567.218
4%	0.07463	13.62700	COP \$547.678
5%	0.06686	12.63749	COP \$529.031
6%	0.06013	11.76126	COP \$511.271

La curva de sensibilidad es decreciente: al aumentar i , disminuye $A_{40:\overline{20}|}^1(i)$ porque los beneficios futuros se descuentan con mayor intensidad. Aunque también disminuye $\ddot{a}_{40:\overline{20}|}(i)$, el efecto sobre el valor del beneficio es mayor y, en consecuencia, la prima neta anual se reduce. Los resultados pueden presentar pequeñas diferencias por el redondeo de la tabla de mortalidad.